



8-Bit Fluids

Erik Buehler • ASTROBi Foundation • erik@astrobi.space
ALife 2026 • Late Breaking Abstract

Dithered Quantization and
Block Floating Point for
Scalable Fluid-Mediated
Artificial Life Simulations



At the microscale, water is both viscous (Stokes) and noisy (Brownian), allowing a CFD solver to trade numerical viscosity and noise for speed. This affords quantizing to **8 bits** per velocity component with little quantization noise impact. Viscosity also makes the field spatially smooth, allowing spectral block floating point to achieve **more than 12.7 effective bits**.

Memory Bandwidth

Computational fluid dynamics (CFD) solvers are typically memory-bound on GPUs. Each pipeline stage reads the full velocity field to global memory and writes it back for the next stage. A 256^3 grid with three fp16 components requires approximately **100 MB** per round-trip per stage. 8-bit storage would save bandwidth, but is usually avoided because it injects quantization noise. Dissipative solvers are generally avoided due to their numerical viscosity (a source of inaccuracy).

In this application, noise and viscosity are desired.

- Cells are **microswimmers**. Their environment is viscous and damped, so energy dissipation is the correct physics rather than a loss of fidelity.
- This environment also exhibits **thermal noise**. A noise-free field would be less realistic than a noisy one.
- Rather than explicitly simulating noise, quantization is used to implicitly provide the noise the simulation needs.

int8 Checkpoint Tap-off and Noise Floor

The dithered int8 quantization at the checkpoint tap-off point does two things: it produces deterministic checkpoints, and it sets the simulation's noise floor, below which BF8's (block 8-bit floating point) encoding error is invisible.

Noise injected every timestep with nothing to remove it would accumulate without bound; dissipation without noise would drive quiescent regions to stasis. But with both, a steady Brownian-like jitter is achieved. One dithered stage is sufficient: the other BF8 stages are undithered, their error sitting below the noise floor.

Why the noise is dithered. Three requirements come from three different directions, and subtractive dither satisfies all three.

- Spectrally flat**, to reduce the numerical artifacts exploitable by evolved agents. Undithered quantization produces intermodulation products. These have structure, are reproducible, and have the regularity digital evolution is known to exploit.
- Whitening spreads low frequency error to higher frequencies** where diffusion attenuates it within a few timesteps.
- Reproducible** for checkpointing. The dither comes from a deterministic generator, so checkpoints are bit-exact.

Spectral Energy Concentration

Energy concentrates at low frequencies, caused by high spatial correlation, and by the fact that most of the fluid volume has low flow velocities. Extra precision is **allocated** to low frequencies, noise (quantization error) is **whitened** to higher frequencies, and dissipation (numerical viscosity) then **attenuates** what landed there.

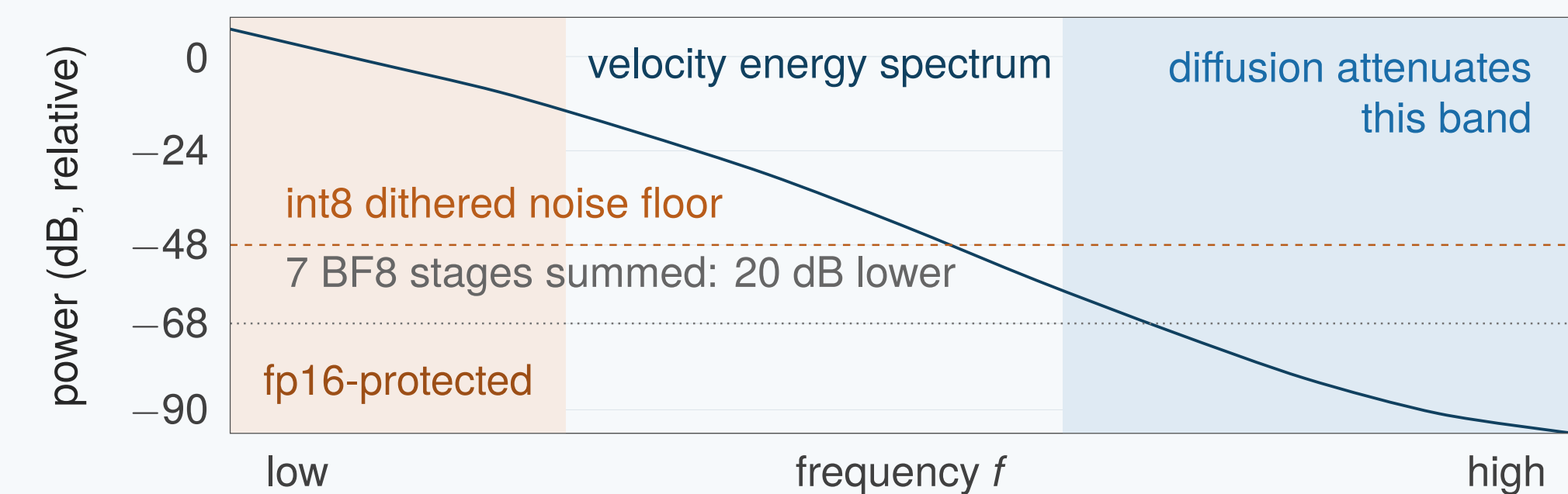
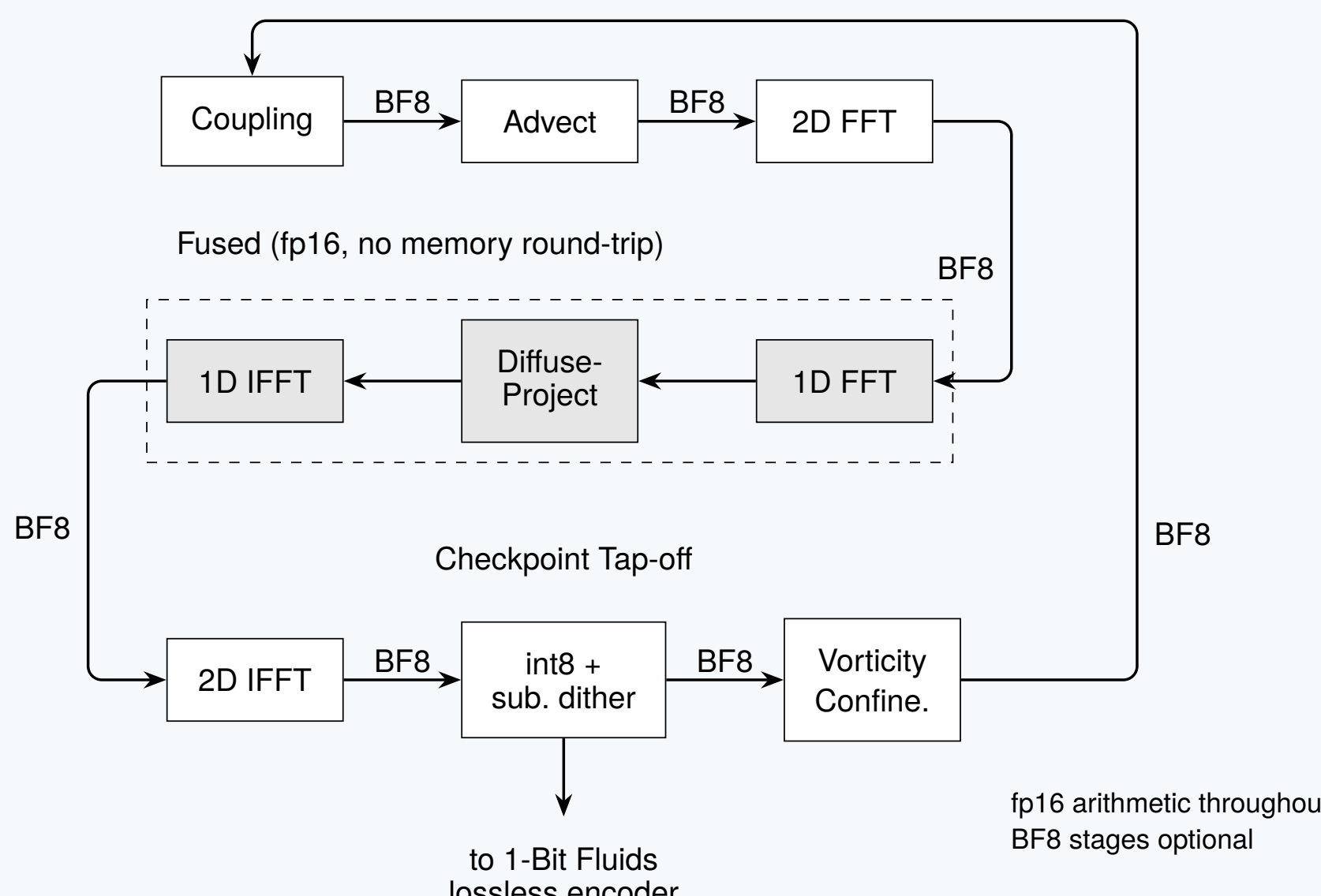


Diagram: the horizontal axis is conceptual, but the dB levels are measured.

This fluid solver is not general purpose, by design. In a high-Reynolds-number application (aerodynamics, etc.), this solver's design choices would be invalid: important energy occupies high frequencies which would get dissipated, and the extra noise would swamp subtle phenomena or qualitatively alter simulation results.

Solver Pipeline

The solver implements Jos Stam's Stable Fluids operator splitting: semi-Lagrangian advection, spectral diffusion, Helmholtz-Hodge projection, and vorticity confinement. All arithmetic is fp16, with fp32 promotion where critical.

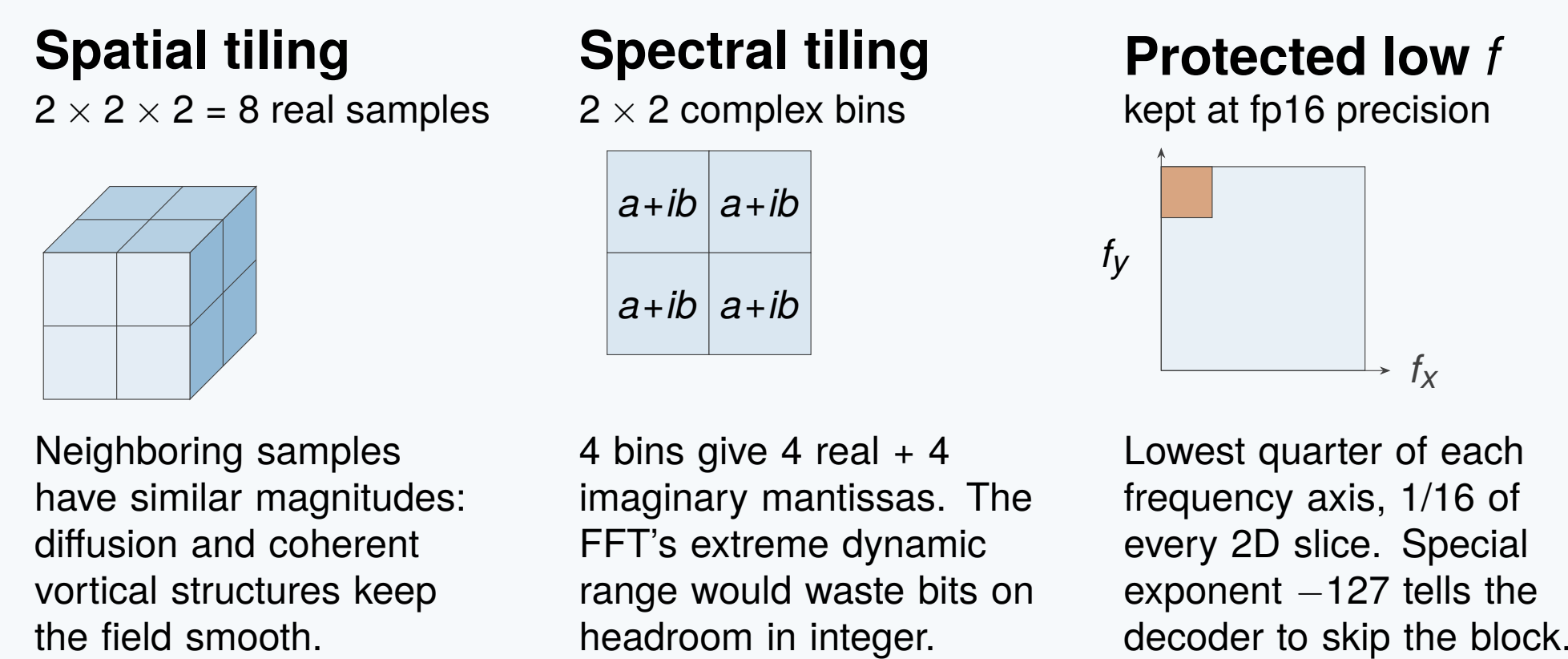
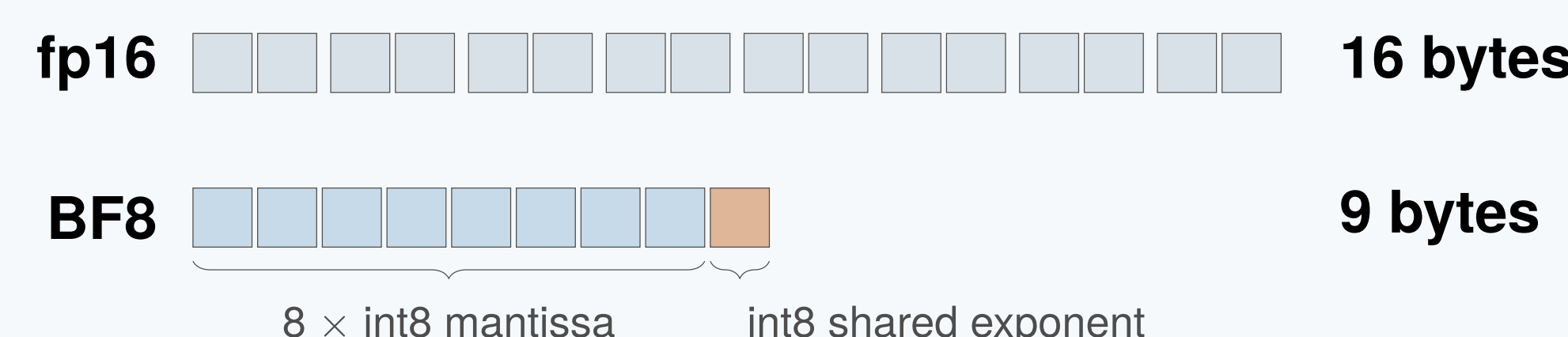


The noise floor is set by the single int8 stage at the tap-off. Each arrow labeled BF8 is a memory round-trip carrying 9 bytes per 8 values instead of 16. The fused spectral section requires no round-trip at all.

Diffusion and projection are fused into a single spectral operation. Vorticity confinement uses spatial finite differences, which was found to be faster than a spectral implementation in testing. A pair of ping-pong buffers serves the whole pipeline: the real-to-complex FFT halves the number of samples but doubles the sample size. Therefore, the amount of data remains fixed so these same ping-pong buffers also work for the spectral data.

BF8 Encoding

A group of 8 values shares a single int8 exponent, with 8 individual int8 mantissas: **9 bytes per block**, 1.125 bytes per value, a $16/9 = 1.78\times$ reduction compared to fp16.



One 9-byte layout with two interpretations. The layout interpretations exploit spatial smoothness and spectral falloff. One encode/decode kernel pair serves the whole pipeline. Encoding and decoding use only integer arithmetic (floor-log2, shift, clamp), which maps to single-cycle GPU instructions.

A uniform BF8 encoding across the entire spectrum reaches 9.9 ENOB (effective number of bits). Keeping the lowest quarter of each spectral axis at fp16 precision protects 1/16 of each 2D slice and raises overall precision to **greater than 12.7 ENOB**. The eight mantissa bytes inside a protected block are then unused, wasting a small amount of memory. Reclaiming these bytes would need a variable-rate layout, and on a bandwidth-bound GPU, simple addressing is worth more than saving a few bytes.

Companion Paper

This paper and a companion paper form a pair. The noise floor exists in part because the checkpoint has to be quantized to int8 to compress well, and BF8 is justifiable because the quant-noise floor exists. The dither required by quantization would inflate compressed checkpoint size by 71% if not for the ECDQ-style synchronization in the *1-Bit Fluids* companion paper.

Noise Budget

The noise budget justifies using BF8 throughout the pipeline.

BF8, single stage	12.7 ENOB / 76.4 dB SNR
BF8, 7 stages summed	11.3 ENOB / 67.9 dB SNR
int8 checkpoint floor	8.0 ENOB / 48.0 dB SNR

With 7 independent BF8 stages, their noise powers sum, reducing the effective SNR to 67.9 dB. This is nearly 20 dB better than the int8 noise floor, so cumulative BF8 noise contributes **less than 1%** of the noise power. By Parseval's theorem, spectral-domain ENOB maps directly to spatial-domain velocity errors, not merely spectral artifacts.

int8 throughout the pipeline was tested and rejected. Quantization noise injected at every stage rapidly overwhelms subtle but important fluid phenomena such as persistent small vortices and wake structures behind moving cells, precisely the phenomena the simulation is intended to exhibit.

Results

The fp16 baseline achieves **190 timesteps per second** for a 256^3 grid (RTX 3090), supporting fluid speeds from near-zero through 1.0 lattice units per step with stable persistent vortices and wake structures. Cost is dominated by the $O(N \log N)$ FFT; cell-fluid coupling scales with cell count, not grid volume.

Timing analysis indicated a significant speedup due to memory bandwidth reductions for all pipeline stages, **except** the hand-implemented 1D FFT. An optimized 1D FFT kernel would make the BF8 pipeline both smaller in memory and faster than the fp16 pipeline.

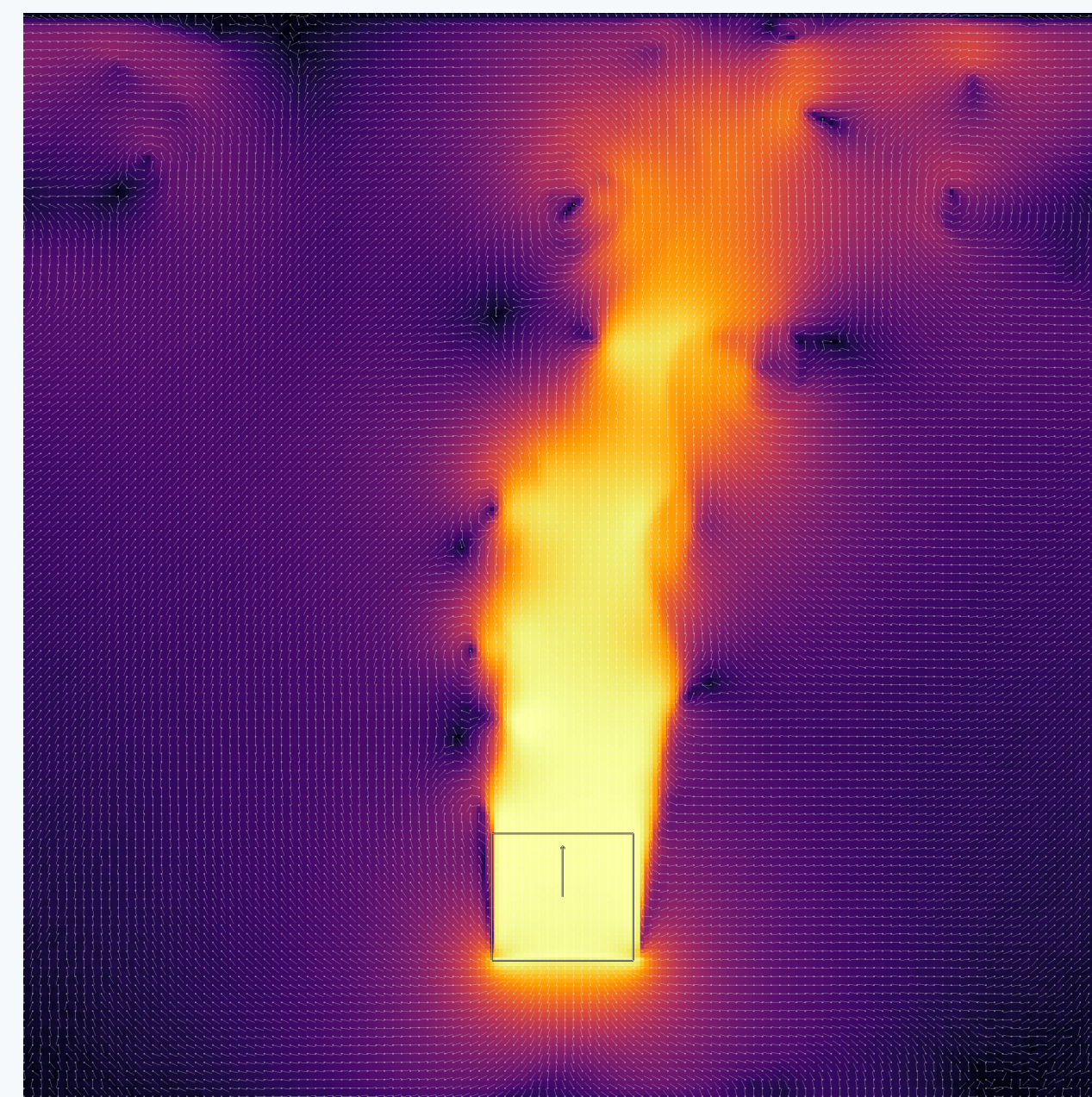
Grid	Voxels	fp16	BF8
256^3	16.8 M	590 MB	449 MB
512^3	134 M	2.8 GB	1.7 GB
$512 \times 1024 \times 1024$	537 M	10.3 GB	5.9 GB
$2048 \times 1024 \times 1024$	2.15 G	*40.4 GB	22.8 GB

GPU memory. The first three fp16 rows are measured via nvtop; *the fourth is estimated and exceeds the RTX 3090's 24 GB. The BF8 column is derived by applying the 16/9 reduction to everything but the fixed CUDA context.

40.4 GB → 22.8 GB

Two billion voxels fit on one 24 GB consumer card. BF8 reduces all velocity storage passing through global memory by a factor of 16/9; fixed overhead such as the CUDA context (approximately 267 MB) is unchanged. Combined with the companion encoder, a 256^3 checkpoint losslessly compresses to **under 6.3 MB**, less than 1 bit per velocity component.

Simulation Screenshot



Velocity magnitude in a 256^3 simulation (XY plane, Y is up). A non-physical velocity source inside the box drives the plume; vanes show flow direction, and the spirals are the recirculation the plume induces. It is a deliberate showpiece rather demonstrating the typical (more viscous) operating point.

References

Stam (1999) *Stable Fluids*. SIGGRAPH. Fedkiw et al. (2001) *Visual simulation of smoke*, SIGGRAPH. Oppenheim (1972) *Realization of digital filters using block-floating-point arithmetic*, IEEE Trans. Audio Electroacoust. Peskin (2002) *The immersed boundary method*, Acta Numerica. Roberts (1962) *Picture coding using pseudo-random noise*, IRE Trans. Inf. Theory. Schuchman (1964) *Dither signals and their effect on quantization noise*, IEEE Trans. Commun. Wannamaker et al. (1997) *A theory of nonsubtractive dither*, IEEE Trans. Signal Process. Lehman et al. (2020) *The surprising creativity of digital evolution*, Artificial Life. Buehler (2026) *1-Bit Fluids* (companion paper), doi.org/10.5281/zenodo.21844937.

© 2026 Erik Buehler. CC BY 4.0. • Video: youtu.be/3dRSOxKgSgg • Full paper: doi.org/10.5281/zenodo.21850231

Acknowledgements

Funding for this work was provided by the ASTROBi Foundation. Anthropic's Claude Opus 4.8 was used to assist with document preparation, porting Python proof-of-concept algorithms to efficient GPU kernels, converting Matplotlib visualizations into OpenGL rendering pipelines, and writing adapter code to interface with pre-compiled open-source entropy encoding and data compression libraries. OpenAI's Codex 5.5 was used to generate extensive test harnesses and to independently review and validate the codebase, avoiding self-referential testing. All algorithmic ideas core to the work were conceived of by the author. All AI-assisted outputs were reviewed, modified where necessary, and validated by the author, who is fully responsible for the final code and manuscript. All references and citations were checked using Google Scholar and verified by the author.